Отчёт по лабораторной работе №8

Математические основы защиты информации и информационной безопасности

Сунь Маосин

Содержание

1 Цель работы **5**

2 Реализация алгоритмов **6**

2.1 Вспомогательные функции преобразования 6

2.1.1 Код функций 6

2.2 Сложение неотрицательных целых чисел 6

2.2.1 Код реализации 7

2.2.2 Результат выполнения 8

2.3 Вычитание неотрицательных целых чисел 8

2.3.1 Код реализации 9

2.3.2 Результат выполнения 10

2.4 Умножение столбиком 10

2.4.1 Код реализации 11

2.4.2 Результат выполнения 12

2.5 Быстрый столбик 12

2.5.1 Код реализации 13

2.5.2 Результат выполнения 14

2.6 Деление многоразрядных чисел 14

2.6.1 Код реализации 16

2.6.2 Результат выполнения 17

2.7 Тестирование всех алгоритмов 17

2.7.1 Код тестирования 19

2.7.2 Полные результаты тестирования 20

3 Вывод **21**

Список иллюстраций

2.1 Вспомогательные функции 6
2.2 Сложение неотрицательных целых чисел 7
2.3 Результат сложения 8
2.4 Вычитание неотрицательных целых чисел 9
2.5 Результат вычитания 10
2.6 Умножение столбиком 11
2.7 Результат умножения столбиком 12
2.8 Быстрый столбик 13
2.9 Результат быстрого умножения 14
2.10 Деление многоразрядных чисел 16
2.11 Результат деления 17
2.12 Функция тестирования 19
2.13 Результаты тестирования всех алгоритмов 20

Список таблиц

1 Цель работы

Изучить и реализовать основные алгоритмы целочисленной арифметики многократной точности. Целью работы является освоение методов выполнения арифметических операций (сложение, вычитание, умножение и деление) над числами, разрядность которых превышает стандартные типы данных ЭВМ, с использованием позиционной системы счисления с основанием $b = 10$.

2 Реализация алгоритмов

2.1 Вспомогательные функции преобразования

Для работы с числами как с массивами цифр были реализованы функции преобразования строк в списки цифр и обратно.

2.1.1 Код функций

```
def str_to_digits(s):
    """Convert string of digits to list of integers"""
    return [int(ch) for ch in s]

def digits_to_str(digits):
    """Convert list of digits to string"""
    return ''.join([str(d) for d in digits])
```

Рис. 2.1: Вспомогательные функции

2.2 Сложение неотрицательных целых чисел

Алгоритм имитирует ручное сложение столбиком. Процесс начинается с младших разрядов и включает перенос k в старшие разряды.

2.2.1 Код реализации

```
def add(u, v, b=10):
    """
    Algorithm 1: Addition of non-negative integers
    u, v: strings of digits (same length)
    b: base (default 10)
    Returns: sum as string
    """
    # Convert strings to lists of digits
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)

    # Initialize
    j = n - 1    # index from rightmost digit (0-based)
    k = 0        # carry
    w_digits = [0] * n

    # Process digits from right to left
    while j >= 0:
        total = u_digits[j] + v_digits[j] + k
        w_digits[j] = total % b
        k = total // b
        j -= 1

    # Add final carry if needed
    if k > 0:
        result = str(k) + digits_to_str(w_digits)
    else:
        result = digits_to_str(w_digits)

    return result
```

Рис. 2.2: Сложение неотрицательных целых чисел

2.2.2 Результат выполнения

```
1. ADDITION
12345 + 56789 = 69134
Expected: 69134
Correct: True
```

Рис. 2.3: Результат сложения

2.3 Вычитание неотрицательных целых чисел

Алгоритм реализует операцию $u - v$ при условии $u > v$. Переменная k используется для фиксации займа из старшего разряда.

2.3.1 Код реализации

```
def subtract(u, v, b=10):
    """
    Algorithm 2: Subtraction of non-negative integers (u > v)
    u, v: strings of digits (same length, u > v)
    b: base (default 10)
    Returns: difference as string
    """
    # Convert strings to lists of digits
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)

    # Initialize
    j = n - 1    # index from rightmost digit
    k = 0          # borrow
    w_digits = [0] * n

    # Process digits from right to left
    while j >= 0:
        diff = u_digits[j] - v_digits[j] + k
        if diff < 0:
            diff += b
            k = -1    # borrow from next digit
        else:
            k = 0
        w_digits[j] = diff
        j -= 1

    # Remove leading zeros
    result = digits_to_str(w_digits).lstrip('0')
    if not result:
        result = '0'

    return result
```

Рис. 2.4: Вычитание неотрицательных целых чисел

2.3.2 Результат выполнения

```
2. SUBTRACTION
56789 - 12345 = 44444
Expected: 44444
Correct: True
```

Рис. 2.5: Результат вычитания

2.4 Умножение столбиком

Классический метод умножения, где каждый разряд множителя v последовательно умножается на все разряды множимого u с последующим суммированием результатов со сдвигом.

2.4.1 Код реализации

```
def multiply_column(u, v, b=10):
    """
    Algorithm 3: Multiplication by column
    u, v: strings of digits
    b: base (default 10)
    Returns: product as string
    """
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)
    m = len(v_digits)

    # Initialize result array
    w_digits = [0] * (m + n)

    # Process each digit of v from right to left
    for j in range(m-1, -1, -1):
        if v_digits[j] == 0:
            continue

        k = 0  # carry
        # Multiply by each digit of u
        for i in range(n-1, -1, -1):
            product = u_digits[i] * v_digits[j] + w_digits[i + j + 1] + k
            w_digits[i + j + 1] = product % b
            k = product // b

        w_digits[j] = k  # final carry for this digit

    # Remove leading zeros
    result = digits_to_str(w_digits).lstrip('0')
    if not result:
        result = '0'

    return result
```

Рис. 2.6: Умножение столбиком

2.4.2 Результат выполнения

```
3. MULTIPLICATION (COLUMN)
123 × 45 = 5535
Expected: 5535
Correct: True
```

Рис. 2.7: Результат умножения столбиком

2.5 Быстрый столбик

Оптимизированный алгоритм умножения, использующий один аккумулятор t для вычисления суммы вкладов всех пар разрядов в конкретную позицию результата.

2.5.1 Код реализации

```
def multiply_fast(u, v, b=10):
    """
    Algorithm 4: Fast multiplication
    u, v: strings of digits
    b: base (default 10)
    Returns: product as string
    """
    u_digits = str_to_digits(u)
    v_digits = str_to_digits(v)
    n = len(u_digits)
    m = len(v_digits)

    # Initialize result array
    w_digits = [0] * (m + n)
    t = 0

    # Process all diagonals
    for s in range(m + n):
        # Sum products along diagonal s
        for i in range(s + 1):
            u_idx = n - 1 - i
            v_idx = m - 1 - (s - i)
            if 0 <= u_idx < n and 0 <= v_idx < m:
                t += u_digits[u_idx] * v_digits[v_idx]

        # Store result digit and keep carry
        w_digits[m + n - 1 - s] = t % b
        t = t // b

    # Remove leading zeros
    result = digits_to_str(w_digits).lstrip('0')
    if not result:
        result = '0'

    return result
```

Рис. 2.8: Быстрый столбик

2.5.2 Результат выполнения

```
4. MULTIPLICATION (FAST)
12345 × 6789 = 83810205
Expected: 83810205
Correct: True
```

Рис. 2.9: Результат быстрого умножения

2.6 Деление многоразрядных чисел

Алгоритм деления реализован методом длинного деления (Long division). Он последовательно обрабатывает цифры делимого, формируя цифры частного и вычисляя остаток.

2.6.1 Код реализации

```
def divide(u, v, b=10):
    """
    Long division algorithm for multi-digit integers
    u, v: strings of digits
    b: base (default 10)
    Returns: (quotient, remainder) as strings
    """
    # Handle division by zero
    if v == "0":
        raise ValueError("Division by zero")

    # If u < v, quotient is 0, remainder is u
    if len(u) < len(v) or (len(u) == len(v) and u < v):
        return "0", u

    # Convert to lists of digits
    u_digits = [int(d) for d in u]
    v_digits = [int(d) for d in v]
    v_len = len(v_digits)

    quotient = []
    remainder = 0

    # Process each digit of u
    i = 0
    while i < len(u_digits):
        # Bring down next digit
        remainder = remainder * b + u_digits[i]

        # Find quotient digit
        q_digit = 0
        while remainder >= int(v):
            remainder -= int(v)
            q_digit += 1

        quotient.append(q_digit)
        i += 1

    # Remove leading zeros from quotient
    while len(quotient) > 1 and quotient[0] == 0:
        quotient.pop(0)

    # Convert to strings
    quotient_str = ''.join(str(d) for d in quotient)
    remainder_str = str(remainder)

    return quotient_str, remainder_str
```


Рис. 2.10: Деление многоразрядных чисел

2.6.2 Результат выполнения

```
5. DIVISION
100 ÷ 7 = 14 remainder 2
Expected: 14 remainder 2
Correct: True
```

Рис. 2.11: Результат деления

2.7 Тестирование всех алгоритмов

Для проверки корректности работы всех алгоритмов была создана функция `test_algorithms`, которая тестирует каждый алгоритм на примерах.

2.7.1 Код тестирования

```
def test_algorithms():
    """Test all algorithms with examples"""

    print("=" * 60)
    print("TESTING MULTIPLE PRECISION ARITHMETIC ALGORITHMS")
    print("=" * 60)

    # Test 1: Addition
    print("\n1. ADDITION")
    u1, v1 = "12345", "56789"
    result1 = add(u1, v1)
    expected1 = str(int(u1) + int(v1))
    print(f"{u1} + {v1} = {result1}")
    print(f"Expected: {expected1}")
    print(f"Correct: {result1 == expected1}")

    # Test 2: Subtraction
    print("\n2. SUBTRACTION")
    u2, v2 = "56789", "12345"
    result2 = subtract(u2, v2)
    expected2 = str(int(u2) - int(v2))
    print(f"{u2} - {v2} = {result2}")
    print(f"Expected: {expected2}")
    print(f"Correct: {result2 == expected2}")

    # Test 3: Multiplication (column)
    print("\n3. MULTIPLICATION (COLUMN)")
    u3, v3 = "123", "45"
    result3 = multiply_column(u3, v3)
    expected3 = str(int(u3) * int(v3))
    print(f"{u3} × {v3} = {result3}")
    print(f"Expected: {expected3}")
    print(f"Correct: {result3 == expected3}")

    # Test 4: Multiplication (fast)
    print("\n4. MULTIPLICATION (FAST)")
    u4, v4 = "12345", "6789"
    result4 = multiply_fast(u4, v4)
    expected4 = str(int(u4) * int(v4))
    print(f"{u4} × {v4} = {result4}")
    print(f"Expected: {expected4}")
    print(f"Correct: {result4 == expected4}")

    # Test 5: Division
    print("\n5. DIVISION")
    u5, v5 = "100", "7"
    q5, r5 = divide(u5, v5)
    expected_q = str(int(u5) // int(v5))
    expected_r = str(int(u5) % int(v5))
    print(f"{u5} ÷ {v5} = {q5} remainder {r5}")
    print(f"Expected: {expected_q} remainder {expected_r}")
    print(f"Correct: {q5 == expected_q and r5 == expected_r}")

    print("\n" + "=" * 60)
```


Рис. 2.12: Функция тестирования

2.7.2 Полные результаты тестирования

```
1. ADDITION
12345 + 56789 = 69134
Expected: 69134
Correct: True

2. SUBTRACTION
56789 - 12345 = 44444
Expected: 44444
Correct: True

3. MULTIPLICATION (COLUMN)
123 × 45 = 5535
Expected: 5535
Correct: True

4. MULTIPLICATION (FAST)
12345 × 6789 = 83810205
Expected: 83810205
Correct: True

5. DIVISION
100 ÷ 7 = 14 remainder 2
Expected: 14 remainder 2
Correct: True
```

Рис. 2.13: Результаты тестирования всех алгоритмов

3 Вывод

В ходе работы были программно реализованы ключевые операции арифметики многократной точности: сложение, вычитание, умножение (классическое и быстрое) и деление. Все алгоритмы успешно протестированы на примерах и работают корректно. Для проверки деления был использован пример $100 \div 7 = 14$ (остаток 2), что подтверждает правильность реализации. Разработанные алгоритмы могут быть использованы для работы с числами произвольной длины, что является важным для криптографических приложений.